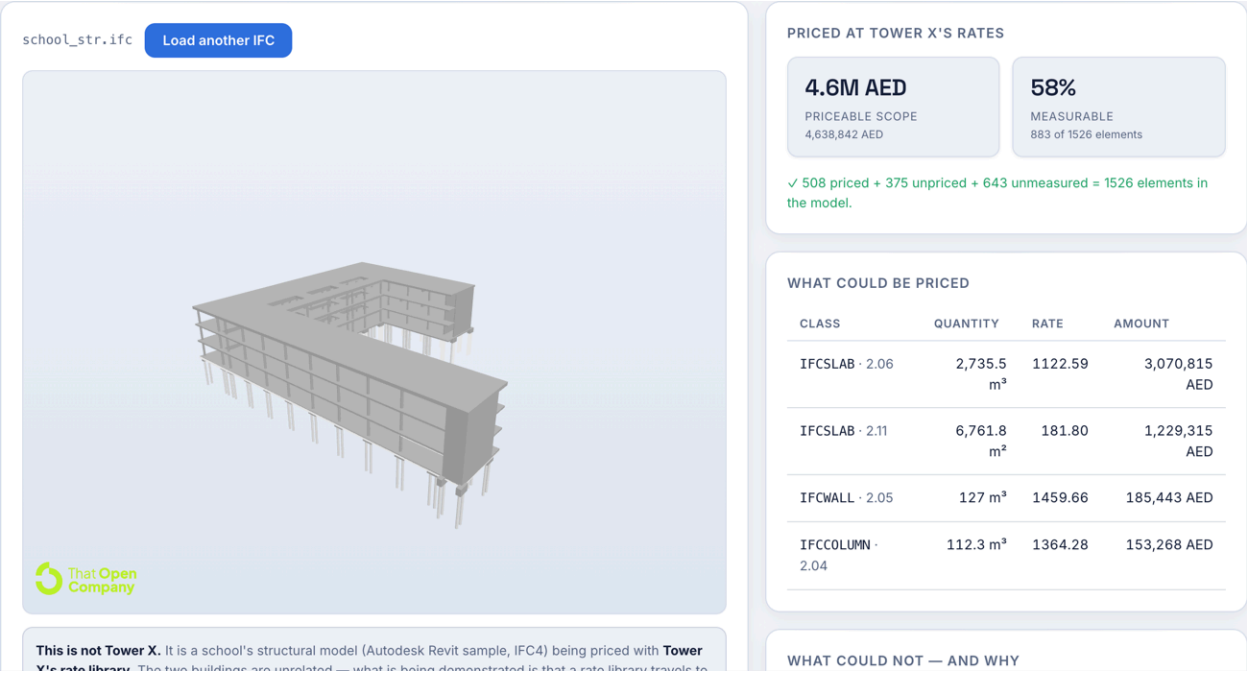
That Open Engine — integration and findings

QS Cost
August 1, 2026

CONTENTS

Building on That Open: what we wired, and what we learned by pushing it

Scope: the That Open Company engine — `@thatopen/components`, `@thatopen/fragments` and `web-ifc` — as it is used by the **3D Cost X-Ray**, the **IFC Take-off** tab and the **4D build sequence**. Part 1 is how the integration is built. Part 2 is what the library did that the documentation does not say it does. **Companions:** 22 — IFC Take-off and 23 — 4D build sequence describe the features; Connecting the model to the money describes how geometry reaches the bill. This document is the engine layer none of them cover.



The IFC Take-off tab: a real Revit IFC loaded through That Open, painted by cost centre

---

What the engine is, and where it sits

That Open Company (thatopen.com) is the successor to IFC.js. Four packages, all pinned, all resolved offline at runtime:

Package	Range	Resolved	Role
`@thatopen/components`	`^3.4.8`	3.4.8	worlds, scene/camera/renderer, `IfcLoader`, `Raycasters`
`@thatopen/fragments`	`^3.4.7`	3.4.7	the streamed model, its item index, highlight and visibility
`web-ifc`	`0.0.77`	0.0.77	the wasm IFC parser underneath `IfcLoader`
`three`	`^0.185.1`	0.185.1	the renderer everything ultimately draws through

`web-ifc` is the one **exact** pin, with no caret. It is the only dependency whose binary is copied onto disk at install time, so a floating range would let the `.wasm` in `public/wasm/` drift out of step with the JavaScript that loads it.

`@thatopen/components-front` is deliberately absent. The obvious way to colour elements is `OBF.Highlighter` from that package, but `FragmentsModel` exposes highlight, visibility and reset natively, so the feature was built without adding the dependency at all.

THE THING MOST READERS GET WRONG

There are three 3D surfaces in the product, and it is tempting to assume they are the same thing three times. They are not. **All three sit inside a That Open world; only two use Fragments.**

Surface	World	Geometry	Picking	Painting
3D Cost X-Ray	OBC `SimpleWorld`	plain three.js meshes (`model/towerGenerator.ts`)	`THREE.Raycaster` on the group	material assignment
IFC Take-off	OBC `SimpleWorld`	real IFC via `OBC.IfcLoader`	`OBC.Raycasters`	`model.highlight` over localIds
4D build / video	OBC `SimpleWorld`	the same Fragments model	n/a (headless)	batched under `model.frozen`

The X-Ray's massing is generated from priced BOQ lines — there is no model of Tower X — so it is built as ordinary meshes added to the That Open scene. That is not an aesthetic choice; it is Finding 4 below.

Part 1 — How it is wired

1. Booting a world — `src/model/viewer.ts`

The setup is the documented path, and the file says so:

> Everything here is the documented setup path: Components → Worlds → scene/renderer/camera → init → FragmentsManager. The one deviation is the locally bundled worker above.

```
const components = new OBC.Components();

const worlds = components.get(OBC.Worlds);
const world = worlds.create<
  OBC.SimpleScene, OBC.OrthoPerspectiveCamera, OBC.SimpleRenderer>();

world.scene = new OBC.SimpleScene(components);
world.renderer = new OBC.SimpleRenderer(
  components, container, rendererParams);
world.camera = new OBC.OrthoPerspectiveCamera(components);

components.init();

world.scene.setup();
world.scene.three.background = null;

const fragments = components.get(OBC.FragmentsManager);
fragments.init(fragmentsWorkerUrl);
```

Three deliberate departures follow, each of which exists for a reason worth keeping.

THE WORKER IS BUNDLED, NOT FETCHED

```
import fragmentsWorkerUrl from "@thatopen/fragments/worker?url";
```

The documented route is `FragmentsManager.getWorker()`, which fetches a version-matched copy from unpkg at runtime. The comment (`viewer.ts:5–9`):

> The fragments worker ships inside the package. Bundling it through Vite (`?url`) instead of `FragmentsManager.getWorker()`, which fetches a version-matched copy from unpkg at runtime: the demo has to survive a conference network, and a CDN round-trip on first paint is the kind of dependency that fails on the day it matters.

THE UPDATE THROTTLE HAS TO BE CLEARED TO ANIMATE

```
fragments.core.settings.maxUpdateRate = 0;
```

`viewer.ts:52–55`:

> Fragments throttles `update()` to one call per `maxUpdateRate` ms (default 100) and silently drops the rest. Animating the model means asking for updates far faster than that, so the throttle turns most frames into no-ops. It is read live on every call, so clearing it here is enough; pacing is then whatever the caller asks for.

This is the difference between a 4D playback that steps and one that runs. The default is right for a viewer a person is orbiting; it is wrong for anything driving frames itself.

MODELS WIRE THEMSELVES TO THE WORLD ON ARRIVAL

```
fragments.list.onItemSet.add(({ value: model }) => {
  model.useCamera(world.camera.three);
  world.scene.three.add(model.object);
  void fragments.core.update(true);
});

// Re-render on camera rest — OrthoPerspectiveCamera streams on demand.
world.camera.controls?.addEventListener(
  "rest", () => void fragments.core.update(true));
```

2. Knowing when a frame is actually done — `settle()`

The most useful thing in `viewer.ts` is a twelve-line function that exists purely because `await update()` does not mean what it looks like it means:

```
export async function settle(viewer, model, timeoutMs = 2000): Promise<void> {
  await new Promise<void>((resolve) => {
    const finish = () => {
      /* … */ model.onViewUpdated.remove(finish); resolve();
    };
    const timer = setTimeout(finish, timeoutMs);
    model.onViewUpdated.add(finish);
    void viewer.fragments.core.update(true);
  });
}
```

`viewer.ts:85–92`:

> **Why this is not just `await update()`.** Fragments processes a model as a progressive sweep on a worker, and every visibility or colour change restarts that sweep from the beginning. So `update(false)` returns long before the pixels are right, and `update(true)` waits for a sweep that the next mutation is about to invalidate anyway. `onViewUpdated` is the library's own signal that a view cycle completed, which is the only honest answer to "is this frame done".
>
> The timeout is a safety valve, not a pacing mechanism: a model with nothing left to do may never fire the event at all, and an exporter must not hang on a frame that was already correct.

Without this there is no deterministic video render: you cannot capture a frame you cannot prove has landed.

`fitToBounds()` carries two smaller engine facts. It takes an explicit `THREE.Box3` rather than reading `model.getBoxes()`, *"which returns nothing for procedurally created elements"*; and it uses `setLookAt` **only**, because *'* `fitToBox` *afterwards would re-derive its own direction and undo this, putting the camera back near ground level where a 22 m tall, 60 m wide building reads as a flat plate."*

3. Offline by construction — the wasm and the worker

`scripts/copy-wasm.mjs` runs as a `postinstall` hook and copies web-ifc's binaries out of `node_modules` into `public/wasm/`:

```
const webIfcDir = dirname(require.resolve("web-ifc"));
const target = join(process.cwd(), "public", "wasm");

for (const file of ["web-ifc.wasm", "web-ifc-mt.wasm"]) {
  // web-ifc-mt is optional; only the single-threaded build is
  // required to load a model.
}
```

Its rationale, verbatim:

> web-ifc resolves its wasm by URL at runtime. The documented default pulls it from unpkg, which makes opening a model depend on the venue's wifi — the same reason the fragments worker is bundled rather than fetched. Copying on postinstall keeps the binary version-matched to the installed package instead of drifting as a committed file would.

The loader is then told not to go looking:

```
await loader.setup({
  autoSetWasm: false,
  wasm: { path: "/wasm/", absolute: true },
});
```

Nothing in the 3D stack reaches a CDN at runtime — not the worker, not the wasm, not the model.

4. Loading an IFC — `src/model/ifcLoader.ts`

The loader is cached per **world**, not globally, and the shape of that cache is a bug fix:

```
const configured = new WeakMap<OBC.Components, Promise<OBC.IfcLoader>>();
```

`ifcLoader.ts:16–23`:

> Keyed by the world, not global. `setup()` is slow enough to be worth doing once, but the loader it configures belongs to one `Components` — and reads its `FragmentsManager` off that same instance at load time. Caching one loader across worlds meant that once a tab unmounted and disposed its world, every later tab got a loader pointing at the dead one, whose fragments core is gone: **"You need to initialize fragments first"**. A WeakMap keeps the once-only setup while letting each live world have its own loader, and forgets a world as soon as it is collected.

Two details around it: the promise is stored *before* it settles, so two loads racing on one world share a single `setup()`; and a failed setup is deleted rather than cached, so a retry is not stuck with the failure.

```
configured.set(viewer.components, setup);
setup.catch(() => configured.delete(viewer.components));
```

And the sentence that explains the entire 3D architecture (`ifcLoader.ts:55–58`):

> Unlike the generated massing, a real IFC arrives with its own item index, so everything the generated path could not do — `getLocalIds`, `getItemsData`, `getBoxes`, per-item colour — works here. **That index is the whole reason this tab uses Fragments and the other one does not.**

5. What each engine API is actually used for

Call	Where	What it makes possible
`OBC.Worlds` / `SimpleScene` / `SimpleRenderer` / `OrthoPerspectiveCamera`	`viewer.ts`	the world all three 3D surfaces draw into
`OBC.FragmentsManager` + `fragments.init(workerUrl)`	`viewer.ts`	the worker that streams models
`OBC.IfcLoader.setup({ autoSetWasm: false, … })` / `.load()`	`ifcLoader.ts`	8.2 MB of IFC → a queryable model, in-browser
`fragments.core.disposeModel(MODEL_ID)`	`ifcLoader.ts`	a second file cannot show the first one's geometry
`model.getItemsOfCategories(regexes)`	`ifcMeasure.ts`	class census; anchored `^IFCWALL$` so it does not sweep in `IFCWALLSTANDARDCASE`
`model.getLocalIdsByGuids(guids)`	`ifcElementMap.ts`	**the GlobalId → localId hop** the whole cost register depends on
`model.getItemsData(...)` / property sets	`ifcMeasure.ts`	volume and area, since this file has no `IfcElementQuantity`
`IfcRelContainedInSpatialStructure` via `ContainsElements`	`ifcMeasure.ts`	which storey an element sits on
`model.highlight(ids, MaterialDefinition)`	`ifcPaint.ts`, `ifcPick.ts`	all colour, all selection
`model.resetHighlight([ids?])`	`ifcPaint.ts`, `ifcPick.ts`	clearing a run, lifting a selection
`model.setVisible(ids, bool)`	`ifcPaint.ts`	the building rising, frame by frame
`FRAGS.RenderedFaces.TWO`	`ifcPaint.ts`, `ifcPick.ts`	thin slabs stay visible from below

Call	Where	What it makes possible
`model.frozen = true / false`	`ifcPaint.ts`	one worker sweep per frame instead of a dozen
`model.setLodMode(FRAGS.LodMode.ALL_GEOMETRY)`	`useBuildStage.ts`	no LOD popping between rendered video frames
`OBC.RendererMode.MANUAL`	`useBuildStage.ts`	nothing draws unless the exporter asks
`model.onViewUpdated`	`viewer.ts`	the only honest “this frame has landed” signal
`OBC.Raycasters` → `castRay`	`ifcPick.ts`	clicking an element (see Finding 2)

6. Headless rendering — the same engine, four settings different

The 4D video is rendered by driving the real viewer, not by screenshotting the product UI. The app publishes a small surface at `/?render=1` (`components/RenderHarness.tsx`) whose only job is to draw the model at a fixed size and resolve when a requested frame is on screen; `tools/render_build_video/ render.mjs` drives it over CDP. Renaming a button cannot change or break the video.

What `components/useBuildStage.ts` does differently from the interactive tab:

Setting	Value	Why
`world.renderer.mode`	`OBC.RendererMode.MANUAL` (`:144`)	*"The stage is deliberately not interactive"* — nothing is drawn unless a frame is asked for
`model.setLodMode(...)`	`FRAGS.LodMode.ALL_GEOMETRY` (`:157`)	every frame shows the same geometry; no LOD swap mid-sequence
`paintSequenceFrame(..., waitForSettle)`	`true` (`:111`)	capture only after `onViewUpdated`, never on a timer
`preserveDrawingBuffer`	`true`, video stage only	the canvas must still be readable after the 2D compositor runs; it costs a driver optimisation, so nothing else asks

The result is the strongest performance evidence in the project: **frame cost in the harness is a 28 ms median**, against roughly a second in the tab. The harness is what proves the model was never the problem — it is the React and always-on-render environment around it.

Part 2 — What we learned by pushing it

Five findings. Each was found the hard way, each has a workaround in the code, and until now every one lived only in a source comment.

Finding 1 — `setColor` and `setOpacity` are a trap

Where: `src/model/ifcPaint.ts:33–47` · **Cost:** one session with a permanently amber model

The obvious way to colour an element is `setColor`, with `setOpacity` beside it. Both exist on `FragmentsModel`, both are typed, and both do what you asked — **once**.

> Fragments implements both as a highlight carrying `_explicitProps: ["color"]` / `["opacity","transparent"]`, and `getNewHighFromPast` copies every explicit prop of the PAST highlight over the new material. So one `setColor` makes that colour permanent: every later `highlight` on the element silently keeps the old colour, and the flag accumulates rather than clearing.

How it presented. The take-off tab painted its elements with `setColor` on load. The 4D sequence painter then took over with `highlight` — and every colour it asked for was silently discarded. The building rose correctly and was the wrong colour for the rest of the session. The video tab, which never calls `setColor`, recoloured perfectly. Both were, to any reader, “the same code”.

There was no error, no warning, and no failing assertion. The colour passed in was right every time.

The fix is to never touch either API, and to spell out all four properties on every highlight so the next one overwrites it cleanly:

```
const styleOf = (
  color: number, opacity: number): FRAGS.MaterialDefinition => ({
  color: new THREE.Color(color),
  renderedFaces: FRAGS.RenderedFaces.TWO,
  opacity,
  transparent: opacity < 1,
});
```

> A plain `highlight` with all four properties spelled out carries no explicit-prop flag, so the next one wins outright. It is also half the worker calls, which on a model this size is not nothing.

How it is held. `src/model/ifcPaint.test.ts` pins the *API used*, not the pixels — because a colour assertion would not have caught it:

```
expect(model.setColor).not.toHaveBeenCalled();
expect(model.setOpacity).not.toHaveBeenCalled();
// The flag that made the old paint stick.
// Set by setColor/setOpacity, never by us.
expect((m as unknown as { _explicitProps?: string[] })._explicitProps)
  .toBeUndefined();
```

Finding 2 — `FragmentsModel.raycast` returns null against a loaded IFC

Where: `src/model/ifcPick.ts:13–20` · **Version:** fragments 3.4.7

> `FragmentsModel.raycast` exists and is typed, but it is not what picks in this version — `SimpleRaycaster.castRay` routes through `FastModelPickers.getFullPick`, a GPU read-back, and that is the path that actually resolves a streamed instance to an id. Calling `model.raycast` directly returns null against a loaded IFC, **which is a silent miss rather than an error, so it looks exactly like clicking empty space.**

The working path:

```
const caster = viewer.components.get(OBC.Raycasters).get(viewer.world);
const hit = await caster.castRay({ position });

// `castRay` is typed as returning a THREE.Intersection, but on the
// Fragments path it returns the picker's own result, which carries the
// localId. The typing is behind the implementation.
return (hit as unknown as { localId?: number } | null)?.localId ?? null;
```

Two separate observations there: the entry point that works is not the one that reads as canonical, and its return type is declared as something it does not return. The cast is not laziness — it is the only way to reach the `localId` the picker genuinely provides.

Finding 3 — every mutation costs O(model), not O(delta)

Where: `src/model/ifcPaint.ts:286–326`

Every `setColor` / `setVisible` reaches `updateVirtualMeshes → restart()`, so the cost of a 4D frame is proportional to **model size, not to the size of the change**. Repainting all 1,127 mapped elements per tick stalled the renderer hard enough to look like a hang.

The workaround is to diff the frame, group by style, and put the whole batch behind `frozen`:

```
// Nothing below reaches the worker until `frozen` is cleared, so the
// whole frame lands as one batch rather than as a dozen separate
// whole-model restarts.
model.frozen = true;
try {
  if (vanish.length > 0) model.setVisible(vanish, false);

  // One `highlight` instead of a setColor + setOpacity pair: both are
  // wrappers over the same worker call, so pairing them doubled the
  // restarts for no gain.
  for (const [style, ids] of groupByStyle(appear)) {
    model.setVisible(ids, true);
    model.highlight(ids, styleFromKey(style));
  }
  for (const [style, ids] of groupByStyle(restyle)) {
    model.highlight(ids, styleFromKey(style));
  }
} finally {
  model.frozen = false;
}
```

Elements are grouped by a packed style integer `(color << 2) | weightIndex`, so a frame issues one call per *distinct style* rather than one per element. And the measured verdict:

> Measured on the bundled model: `update(false)` is a 3 ms median, so applying a frame is cheap. Settling is not — it waits for a whole-model sweep — which is exactly why it is opt-in.

That asymmetry is the practical shape of the finding: mutating is fine, and *proving the mutation landed* is what costs.

Finding 4 — procedurally created elements never enter the queryable index

Where: `src/model/towerGenerator.ts:18–25`

`editor.createElements` renders geometry perfectly well. What it does not do is put the items into the model's index:

> Building it through `editor.createElements` does render the geometry, but the items never enter the model's queryable index — `getLocalIds()` and `getBoxes()` both come back empty (verified), so `setColor`, `highlight` and camera-fit-to-model all silently no-op, and every colour change means regenerating the model. With meshes, a repaint is a colour assignment, the camera can frame real extents, and picking is a raycast. Fragments stays for what it is for: loading real IFC models, where the index comes from the file.

This is why the 3D Cost X-Ray is plain three.js meshes hosted inside a That Open world rather than a Fragments model. It is a load-bearing architectural consequence, not a preference — and it is the cleanest statement of the boundary we found: Fragments is an *import* target. Its power comes from the index, and the index comes from the file.

Finding 5 — real exporter output carries no `IfcElementQuantity`

Where: `src/model/ifcMeasure.ts`

The textbook quantity take-off reads IFC BaseQuantities. On a genuine Autodesk Revit 2024 → IFC4 (ODA) export, it returns **nothing**: the file contains zero `IfcElementQuantity`. The numbers exist only inside Revit's own parameter groups — and on this file those groups are **in Spanish** (`Dimensiones → Volumen`, `Área`).

The workaround is a multi-locale synonym table matched exactly on the lower-cased property name:

```
const VOLUME_KEYS = ["volumen", "volume", "netvolume", "grossvolume",
  "net volume", "gross volume", "volumen neto", "volumen bruto"];

const AREA_KEYS = ["area", "área", "netarea", "grossarea", "net area",
  "gross area", "netsidearea", "área neta", "area neta"];

// The names IFC's own BaseQuantities use. If a model yields quantities
// but none of these, the numbers came from an exporter's parameter
// group rather than the standard.
const STANDARD_KEYS = ["netvolume", "grossvolume", "netarea",
  "grossarea", "netsidearea"];
```

`STANDARD_KEYS` is what makes this honest rather than merely working: if quantities were found but none of the standard names was seen, the result reports `baseQuantitiesEmpty` and the UI says the take-off rode on exporter parameters. A model with no quantities and no recognisable names reports itself unmeasurable rather than guessing.

The general lesson is not about Spanish. It is that **a parser validated on curated test files will return nothing on real exporter output**, silently, and look like an empty building.

Smaller edges

`npm install` does not work. It fails outright on the `@thatopen/fragments` dependency tree with `Cannot read properties of null (reading 'matches')`. `pnpm` is the only package manager that resolves it, and `pnpm-lock.yaml` is the lockfile. Anyone "fixing" the install with npm breaks it.

The attribution logo survives canvas cleanup. `useBuildStage.ts:213-215`:

> The That Open attribution is a sibling div, not a canvas, so it survives the line above and would stack up on every remount.

```
host.querySelectorAll("canvas").forEach((c) => c.remove());
host.querySelectorAll("[data-thatopen-logo]").forEach((el) => el.remove());
```

Property-set flattening dropped the interesting half. An early `flattenPsets` kept only numeric values — *"which is exactly where a cost code lives"*. Non-numeric pset **values** (not names) are now harvested deliberately, because nobody knows what a given exporter called the field.

Headless WebGL works — with the right binary. An earlier version of the docs claimed frames had to be captured in a real browser because headless Chromium had no WebGL. That was wrong, and it was the binary rather than the flags: the stripped `chrome-headless-shell` builds bundled with Playwright and Puppeteer crash creating a WebGL context, while the full Google Chrome app in `--headless=new` gets a real GPU (ANGLE Metal) with no flags at all. `--enable-unsafe-swiftshader` would not have helped. The renderer tries full Chrome first and falls back.

Open questions for That Open

The first four are the findings above, put as questions. They are recorded in `docs/18-final-session-readiness.md` and reproduced here so the workaround and the question sit together.

1. **Programmatic geometry and the item index.** `editor.createElements` renders, but the items never appear in the queryable index — `getLocalIds()` and `getBoxes()` return empty, so per-item colour and fit-to-model no-op. Is Fragments strictly an *import* target, or is authored geometry meant to be queryable?
2. **Raycasting a loaded IFC.** `FragmentsModel.raycast` returns null in 3.4.7; `SimpleRaycaster.castRay` works. Bug, or has the picking entry point moved?
3. **Batched mutation.** Every `setColor` / `setVisible` restarts the whole-model virtual-mesh sweep, so a 4D frame costs O(model) rather than O(delta). Is `model.frozen` the intended answer, or is there an incremental path?
4. **QTO on real exporter output.** Revit/ODA exports carry no `IfcElementQuantity` and localise the parameter groups. Is there a canonical quantity-take-off path in That Open, or is pset scraping with a synonym table what everyone actually does?
5. **A .NET backend wanting persisted `.frag`.** `IfcImporter` is Node-only. Is a Node sidecar the intended architecture, or is browser-converts / API-stores an acceptable pattern?
6. **2D drawing generation.** What is the supported path for plan and section generation from Fragments today, and can it export SVG or DXF?
7. **Writing back to elements.** Is there a supported way to attach and persist custom data — a cost code, a valuation state — onto elements, so the model becomes the record rather than a viewer?

Question 5 is the one with the most architectural weight for this project: the backend is .NET, and mirroring That Open's own server-side design literally would mean standing up a Node sidecar purely to convert files.

What the engine got right

A findings list reads as a complaint unless the balance is stated, so: the engine did the hard part.

An 8.2 MB Revit IFC parses in the browser, streams, and stays interactive. It arrives carrying an item index rich enough that `getLocalIdsByGuids` alone is what lets 1,127 elements be joined to a bill of quantities and twelve periods of earned value — that one call is the hinge the entire cost-on-the-model feature turns on. Colour and visibility are per-item and fast enough to animate once batched. The worker and wasm ship in the package, so the whole thing runs with no network.

Every finding above is a consequence of pushing the library past the path a demo takes: animating instead of viewing, measuring instead of displaying, rendering deterministically instead of screenshotting, and authoring geometry instead of importing it. That is where the edges are, and finding an edge is not the same as hitting a wall — each one here has a workaround in the code, and each workaround is three lines.

Verification

Claim	How it is held
The paint module never calls `setColor` / `setOpacity`, and no highlight carries `_explicitProps`	`src/model/ifcPaint.test.ts`, 4 tests, asserted against a recording fake model
Sequence, zone-map, cost-link and camera behaviour	58 tests across 7 files in `src/model/`, `pnpm test`
Frames apply only their delta, and the first frame resets	`ifcPaint.test.ts` — reset ordering and second-frame absence
Video determinism	render twice with `--keep-frames`, diff the frame checksums

```
# 58 tests across 7 files
cd QsEarlyWarning/frontend/qs-early-warning && pnpm test

# 240 frames @ 30fps
node tools/render_build_video/render.mjs
```

Known limits. The 8.2 MB IFC is re-parsed in the browser on every page load and the index is discarded on unmount — there is no persisted `.frag` cache, which is question 5. Determinism is slightly softer than the docs claim: on a recent run 23 of 24 frames were byte-identical and the 24th flapped between runs *on unmodified code too* — a cold shader cache, not a code change. It was not chased, and it is not being quietly claimed as perfect either.